Question_For_LAB_5

1. Write a C program to:
    - Create a TCP Socket.

    Client: Send TWO SORTED Array to the Server.
    Server: Merge the two array, and return back to the Client.

    Goto: Program5_1

2. Write a C program to:
    - Create a TCP Socket.

    Client: Send TWO INTEGER and ONE SYMBOL (+, -, *, /).
    Server: It will conduct the mathematical operation on the two number and return the result to the client.

    Goto: Program5_2

3. Write a C program to:
    - Create a TCP Socket.

    Client: Send a DOMAIN NAME to the Server.
    Server: Returns the IP of that domain back to the Client.

    Goto: Program5_3

4. Write a C program to:
    - Create a TCP Socket.

    Client: Send the LIST COMMAND to the Server.
    Server: It will return back the list of file names in the present working directory to the Client.

    Client: Send ‘ retr <file_name> ’

Server: Sends back the content of the particular file.

Goto: Program5_4

Program5_1

1. Write a C program to:
    - Create a TCP Socket.

    Client: Send TWO SORTED Array to the Server.

    Server: Merge the two array, and return back to the Client.

TCP_Server.c

```c
// Receive TWO SORTED Arrays from Client
// Merge them and send the result back

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define MAX 100

int main() {
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_size = sizeof(client_addr);

    int n1, n2;
    int a[MAX], b[MAX], merged[MAX * 2];

    // Creating a SOCKET
    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    // Server address
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    // Bind
    bind(server_fd, (struct sockaddr *)&server_addr,
                    sizeof(server_addr));

    // Listen
    listen(server_fd, 5);

    printf("Server waiting for connection...\n");
```

```c
// Accept Client
client_fd = accept(server_fd,
                   (struct sockaddr *)&client_addr,
                   &addr_size);

printf("Client Connected!\n");

// Receive sizes
recv(client_fd, &n1, sizeof(n1), 0);
recv(client_fd, &n2, sizeof(n2), 0);

// Receive arrays
recv(client_fd, a, n1 * sizeof(int), 0);
recv(client_fd, b, n2 * sizeof(int), 0);

// Merge
int i = 0, j = 0, k = 0;

while (i < n1 && j < n2) {
    if (a[i] < b[j])
        merged[k++] = a[i++];
    else
        merged[k++] = b[j++];
}

while (i < n1)
merged[k++] = a[i++];

while (j < n2)
    merged[k++] = b[j++];

// Send size
int total = n1 + n2;
send(client_fd, &total, sizeof(total), 0);

// Send merged array
send(client_fd, merged, total * sizeof(int), 0);

printf("Merged array sent to Client.\n");
```

```
    close(client_fd);
    close(server_fd);

    return 0;
}
```

TCP_Client.c

```c
// Send TWO SORTED Array to the Server.

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<unistd.h>
#include<arpa/inet.h>

#define PORT 8080
#define MAX 100

int main(){
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_size = sizeof(client_addr);

    int n1, n2;
    int a[MAX], b[MAX], merged[MAX * 2];

    // Creating a SOCKET
    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    // Server address
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    // Bind
    bind(server_fd, (struct sockaddr *)&server_addr,
sizeof(server_addr));

    // As this is a TCP Server, Listen
    listen(server_fd, 5);
    printf("Server waiting for connection...\n");

    // Accept the Client
    client_fd = accept(server_fd,
```

```
                            (struct sockaddr
*)&client_addr,
                            &addr_size);
    printf("Client Connected!\n");

    // Receive sizes
    recv(client_fd, &n1, sizeof(n1), 0);
    recv(client_fd, &n2, sizeof(n2), 0);

    // Receive Array
    recv(client_fd, a, n1 * sizeof(int), 0);
    recv(client_fd, b, n2 * sizeof(int), 0);

    // Merge the TWO SORTED Array
    int i = 0, j = 0, k = 0;

    while (i < n1 && j < n2){
        if (a[i] < b[j])
            merged[k++] = a[i++];
        else
            merged[k++] = b[j++];
    }

    while (i < n1)
        merged[k++] = a[i++];

    while (j < n2)
        merged[k++] = b[j++];

    // Send merged SIZE
    int total = n1 + n2;
    send(client_fd, &total, sizeof(total), 0);

    // Send merged ARRAY
    send(client_fd, merged, total * sizeof(int), 0);
    printf("Merged array sent to Clinet\n");

    close(client_fd);
    close(server_fd);
```

```
    return 0;
}
```

Program5_2

2. Write a C program to:

- Create a TCP Socket.

Client: Send TWO INTEGER and ONE SYMBOL (+, -, *, /).

Server: It will conduct the mathematical operation on the two number and return the result to the client.

TCP_Server.c

```c
// Receive TWO INTEGER and ONE SYMBOL from Client
// Perform the operation and return result

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8081

int main() {
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_size = sizeof(client_addr);

    int a, b;
    char op;
    float result;

    // Creating SOCKET
    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    // Server address
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    // Bind
    bind(server_fd, (struct sockaddr *)&server_addr,
         sizeof(server_addr));

    // Listen
    listen(server_fd, 5);

    printf("Server waiting for connection...\n");
```

```c
// Accept Client
client_fd = accept(server_fd,
                   (struct sockaddr *)&client_addr,
                   &addr_size);

printf("Client Connected!\n");

// Receive data
recv(client_fd, &a, sizeof(a), 0);
recv(client_fd, &b, sizeof(b), 0);
recv(client_fd, &op, sizeof(op), 0);

// Perform operation
switch (op) {
    case '+':
        result = a + b;
        break;

    case '-':
        result = a - b;
        break;

    case '*':
        result = a * b;
        break;

    case '/':
        if (b == 0) {
            result = 0;
            printf("Division by zero!\n");
        } else {
            result = (float)a / b;
        }
        break;

    default:
        result = 0;
        printf("Invalid operator!\n");
}
```

```
    // Send result
    send(client_fd, &result, sizeof(result), 0);

    printf("Result sent to Client.\n");

    close(client_fd);
    close(server_fd);

    return 0;
}
```

TCP_Client.c

```c
// Send TWO INTEGER and ONE SYMBOL to the Server

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8081

int main() {
    int sock;
    struct sockaddr_in server_addr;

    int a, b;
    char op;
    float result;

    // Creating SOCKET
    sock = socket(AF_INET, SOCK_STREAM, 0);

    // Server address
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    server_addr.sin_addr.s_addr =
inet_addr("127.0.0.1");

    // Connect
    connect(sock, (struct sockaddr *)&server_addr,
            sizeof(server_addr));

    printf("Connected to Server!\n");

    // Input
    printf("Enter first integer: ");
    scanf("%d", &a);
```

```
    printf("Enter second integer: ");
    scanf("%d", &b);

    printf("Enter operator (+, -, *, /): ");
    scanf(" %c", &op);

    // Send data
    send(sock, &a, sizeof(a), 0);
    send(sock, &b, sizeof(b), 0);
    send(sock, &op, sizeof(op), 0);

    // Receive result
    recv(sock, &result, sizeof(result), 0);

    printf("Result = %.2f\n", result);

    close(sock);

    return 0;
}
```

Program5_3

3. Write a C program to:
    - Create a TCP Socket.

    Client: Send a DOMAIN NAME to the Server.

    Server: Returns the IP of that domain back to the Client.

TCP_Server.c

```c
// Receive DOMAIN NAME from Client
// Return IP address of the domain

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <netdb.h>

#define PORT 8082
#define MAX 100

int main() {
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_size = sizeof(client_addr);

    char domain[MAX];
    char ip[MAX];

    // Creating SOCKET
    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    // Server address
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    // Bind
    bind(server_fd, (struct sockaddr *)&server_addr,
         sizeof(server_addr));

    // Listen
    listen(server_fd, 5);

    printf("Server waiting for connection...\n");
```

```
    // Accept Client
    client_fd = accept(server_fd,
                       (struct sockaddr *)&client_addr,
                       &addr_size);

    printf("Client Connected!\n");

    // Receive domain
    recv(client_fd, domain, sizeof(domain), 0);

    printf("Domain received: %s\n", domain);

    // DNS lookup
    struct hostent *host = gethostbyname(domain);

    if (host == NULL) {
        strcpy(ip, "Unable to resolve domain");
    } else {
        struct in_addr **addr_list =
            (struct in_addr **)host->h_addr_list;

        if (addr_list[0] != NULL)
            strcpy(ip, inet_ntoa(*addr_list[0]));
        else
            strcpy(ip, "IP not found");
    }

    // Send IP
    send(client_fd, ip, sizeof(ip), 0);

    printf("IP sent to Client.\n");

    close(client_fd);
    close(server_fd);

    return 0;
}
```

TCP_Client.c

```c
// Send DOMAIN NAME to Server

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8082
#define MAX 100

int main() {
    int sock;
    struct sockaddr_in server_addr;

    char domain[MAX];
    char ip[MAX];

    // Creating SOCKET
    sock = socket(AF_INET, SOCK_STREAM, 0);

    // Server address
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    server_addr.sin_addr.s_addr =
inet_addr("127.0.0.1");

    // Connect
    connect(sock, (struct sockaddr *)&server_addr,
            sizeof(server_addr));

    printf("Connected to Server!\n");

    // Input domain
    printf("Enter domain name: ");
    scanf("%s", domain);
```

```
    // Send domain
    send(sock, domain, sizeof(domain), 0);

    // Receive IP
    recv(sock, ip, sizeof(ip), 0);

    printf("IP Address: %s\n", ip);

    close(sock);

    return 0;
}
```

Program5_4

4. Write a C program to:
    - Create a TCP Socket.

    Client: Send the LIST COMMAND to the Server.
    Server: It will return back the list of file names in the present working directory to the Client.

    Client: Send ‘ retr <file_name> ’
    Server: Sends back the content of the particular file.

GOTO - Client
Server

TCP_Server.c

```c
// LIST: Send file names in present working directory
// retr <file_name>: Send contents of requested file

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <dirent.h>

#define PORT 8083
#define MAX 1024

int main() {
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_size = sizeof(client_addr);

    char command[MAX];
    char buffer[MAX];

    // Creating SOCKET
    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    // Server address
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    // Bind
    bind(server_fd, (struct sockaddr *)&server_addr,
         sizeof(server_addr));

    // Listen
    listen(server_fd, 5);

    printf("Server waiting for connection...\n");
```

```
    // Accept Client
    client_fd = accept(server_fd,
                       (struct sockaddr *)&client_addr,
                       &addr_size);

    printf("Client Connected!\n");

    // Receive command
    memset(command, 0, sizeof(command));
    recv(client_fd, command, sizeof(command) - 1, 0);

    // LIST command
    if (strcmp(command, "LIST") == 0) {

        DIR *dir;
        struct dirent *entry;

        dir = opendir(".");

        if (dir == NULL) {
            strcpy(buffer, "Unable to open
directory.");
            send(client_fd, buffer, strlen(buffer) + 1,
0);
        } else {

            memset(buffer, 0, sizeof(buffer));

            while ((entry = readdir(dir)) != NULL) {

                // Ignore . and ..
                if (strcmp(entry->d_name, ".") != 0 &&
                    strcmp(entry->d_name, "..") != 0) {

                    strcat(buffer, entry->d_name);
                    strcat(buffer, "\n");
                }
            }
```

```
            closedir(dir);

            send(client_fd, buffer, strlen(buffer) + 1,
0);
            printf("Directory list sent to Client.\n");
        }
    }

    // RETR command
    else if (strncmp(command, "retr ", 5) == 0) {

        char filename[MAX];

        strcpy(filename, command + 5);

        FILE *fp = fopen(filename, "r");

        if (fp == NULL) {
            strcpy(buffer, "ERROR: File could not be
opened.");
            send(client_fd, buffer, strlen(buffer) + 1,
0);
        } else {

            while (fgets(buffer, sizeof(buffer), fp) !=
NULL) {
                send(client_fd, buffer, strlen(buffer),
0);
            }

            // End marker
            strcpy(buffer, "\n---END OF FILE---\n");
            send(client_fd, buffer, strlen(buffer), 0);

            fclose(fp);

            printf("File content sent to Client.\n");
        }
    }
```

```
    else {
        strcpy(buffer, "Invalid command.");
        send(client_fd, buffer, strlen(buffer) + 1, 0);
    }

    close(client_fd);
    close(server_fd);

    return 0;
}
```

TCP_Client.c

```c
// Send LIST command or retr <file_name> to Server

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8083
#define MAX 1024

int main() {
    int sock;
    struct sockaddr_in server_addr;

    char command[MAX];
    char buffer[MAX];

    // Creating SOCKET
    sock = socket(AF_INET, SOCK_STREAM, 0);

    // Server address
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    server_addr.sin_addr.s_addr =
inet_addr("127.0.0.1");

    // Connect
    connect(sock, (struct sockaddr *)&server_addr,
            sizeof(server_addr));

    printf("Connected to Server!\n");

    // Send LIST command
    printf("Enter command (LIST / retr <file_name>):
");
    fgets(command, sizeof(command), stdin);
```

```
    // Remove newline
    command[strcspn(command, "\n")] = '\0';

    send(sock, command, strlen(command) + 1, 0);

    // Receive response
    printf("\nServer Response:\n");

    int bytes;

    while ((bytes = recv(sock, buffer,
                         sizeof(buffer) - 1, 0)) > 0) {

        buffer[bytes] = '\0';

        printf("%s", buffer);

        // Stop after end marker
        if (strstr(buffer, "---END OF FILE---") !=
NULL)
            break;

        // LIST response normally arrives as one
message
        if (strcmp(command, "LIST") == 0)
            break;
    }

    printf("\n");

    close(sock);

    return 0;
}
```